

LABORATORIO 3: TDD – Pruebas de Software – Agilismo y Scrum – Análisis de Requerimientos

Escuela colombiana de ingeniería – CVDS/DOSW

OBJETIVOS:

En el presente laboratorio vamos a aplicar el manejo de TDD para proyectos de software, junto a la importancia de las pruebas unitarias, todo esto en un caso de estudio práctico. Para esto se utilizará JUnit para construir las pruebas, medir cobertura de código con Jacoco y analizar la calidad de código con SonarQube.

La idea central es que los tres integrantes aprendan a trabajar con tareas divididas, distribuyendo el flujo de trabajo adecuadamente aplicando metodologías Ágiles y analizando los requerimientos necesarios, todo en pro de fortalecer el trabajo en equipo.

PRE-REQUISITOS:

- Java OpenJDK Runtime Environment: 17.x.x
- Apache Maven: 3.9.x
- JUnit: 5.x.x
- Git y una cuenta de GitHub
- Docker

PARTE 2 – Hora del Código:

Para esta sección los retos se tendrán que hacer de forma secuencial, es decir para cumplir con el reto 2 es necesario que el reto 1 este culminado, y así sucesivamente, importante que tengan en cuenta que es necesario analizar y diseñar.

Tienen el tiempo de la clase para resolver la mayor cantidad de retos posibles. Los retos faltantes deberán ser completados durante la semana.

Cada integrante del equipo debe participar en cada reto evidenciando el commit de su colaboración: “Parte 2 – Reto #N : Nombre Integrante - Acción realizada”.

BANKIFY

Una startup fintech llamada **Bankify** está desarrollando un sistema de gestión de cuentas bancarias y servicios financieros básicos. La empresa necesita asegurar que su software cumpla con los más altos estándares de calidad, seguridad y mantenibilidad, aplicando metodologías ágiles, TDD, pruebas automatizadas, cobertura de código (JaCoCo) y análisis estático con SonarQube.



El producto inicial contempla que los números de cuenta deben tener exactamente 10 dígitos, Solo son válidos si los dos primeros dígitos corresponden a un banco registrado (ejemplo 01 BANCOLOMBIA, 02 DAVIVIENDA), Cada cuenta no debe tener letras ni caracteres especiales.

El software debe permitir crear y validar las cuentas de los clientes, permitir que se realice la consulta del saldo de una cuenta y hacer un depósito.

RETO #1: Identificando los Requerimientos

A partir de la lectura del caso fintech su misión es:

- Identifiquen reglas de negocio.
- Definan las funcionalidades principales.
- Escriban los actores principales.
- Documenten las precondiciones necesarias para el sistema.

Entregable: Información contenida en el README.md general

RETO #2: Diseñando

A partir del punto anterior, su misión es la siguiente:

- Realicen un diagrama de contexto de la solución (relaciones entre el sistema y los actores identificados) y explíquenlo.
- Realicen un diagrama de Casos de uso en base a sus actores y las funcionalidades identificadas.
- En base al diagrama anterior generen historias de usuario basado en el formato: Como [actor], quiero [objetivo], para [beneficio]
- Generen una tabla de Excel con las historias de usuario creadas e indique para cada una que atributo de calidad resuelve.
- Realicen un diagrama de Clases de las entidades identificadas en el enunciado.

Entregable: Diagramas UML en la paquetería adecuada, Evidencias en el README.md

RETO #3: Una estimación automatizada

Por orden del área de Producto se quiere automatizar el consenso y decisión de las estimaciones para las tareas y desean que implementen:

- Una aplicación que por medio de la consola simule una sesión de Planning Poker.
- El programa debe recibir las tareas creadas en el RETO 2 y para cada historia el programa debe pedir a los integrantes del equipo que voten usando la secuencia de Fibonacci (1,2,3,5,8,13).

- Si todos los votos son iguales se asigna ese puntaje y se pasa a la siguiente historia.
- Si los votos son diferentes, el programa debe mostrar los votos y decir “Votos divergentes – Discutan y vuelvan a votar” y repetir la votación para esa historia hasta que haya consenso.
- Al final el programa debe mostrar un resumen con cada historia y su puntaje final.

Entregable:

- Código de la simulación en su respectiva carpeta
- Evidencia de la implementación en código en el README.md
- Patrones y Principios Utilizados – Explicación de cada uno en el README.md
- Captura de Pantalla de la Evidencia de las votaciones mostrando todos los casos y puntos finales para las historias en el README.md

RETO #4: Tiempo De Desarrollo

A partir de los retos anteriores y en equipo:

- Creen las clases modelo principales con sus atributos, sus métodos respectivos (getters y setters)
- Implementen la clase que se encargue de Validar las Cuentas aplicando el ciclo TDD (Rojo -> Verde -> Refactor)
 - Primero creen pruebas unitarias con Junit 5
 - Usen Streams y Lambdas para mejorar el código
 - Cada método y clase debe tener su respectivo JAVA Doc.
- Implementen las clases que se encarguen de Gestionar las Cuentas (creación, realizar depósitos y consulta) aplicando el ciclo TDD (Rojo -> Verde -> Refactor)
 - Primero creen pruebas unitarias con Junit 5
 - Usen Streams y Lambdas para mejorar el código
 - Cada método y clase debe tener su respectivo JAVA Doc.
- Corran las Pruebas y asegúrense de que todas Pasen.

Entregable:

- Código Fuente de las clases e implementaciones
- Código Fuente de las clases de pruebas.
- Evidencia en el README.md de la ejecución exitosa de las pruebas
- Explicación de que principios y patrones utilizaron para la solución en el README.md

RETO #5: Corran que ahí viene El JaCOCO

La startup fintech está satisfecha con su trabajo anterior, pero ahora quiere asegurarse de que la calidad del código sea medible y se mantenga alta. La misión de ustedes como equipo es la siguiente:

- Actualicen el plugin de jacoco de la siguiente manera para tener un porcentaje mínimo de cobertura de un 85%

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.12</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
          <configuration>
            <excludes>
              <exclude>/configurators</exclude>
            </excludes>
          </configuration>
        </execution>
        <execution>
          <id>jacoco-check</id>
          <goals>
            <goal>check</goal>
          </goals>
          <configuration>
            <rules>
              <rule>
                <element>PACKAGE</element>
                <limits>
                  <limit>
                    <counter>CLASS</counter>
                    <value>COVEREDRATIO</value>
                    <minimum>0.85</minimum><!--Porcentaje mínimo de cubrimiento para construir el proyecto-->
                  </limit>
                </limits>
              </rule>
            </rules>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

- Compilen el proyecto, al realizarlo en la carpeta target se debe crear una carpeta con el nombre site/jacoco la cual tiene un index.html ese es su reporte de la cobertura, ahí se puede visualizar la cobertura total y de cada una de las clases.
- Tome una captura del archivo y adjúntela al README.md como cobertura inicial, ¿Cuál es el porcentaje de cobertura de instrucciones y de ramas? ¿Hay alguna línea o condición que las pruebas no cubrieron?
- Si la cobertura no es mayor al 85% añadan los casos de prueba que hagan falta para alcanzar la cobertura, ejecuten de nuevo y tomen evidencia.

Entregable:

- Evidencias en el README.md principal, en la sección del reto
- Escriban una breve reflexión sobre qué casos de prueba tuvieron que añadir para mejorar la cobertura y porque es importante esa métrica.

RETO #6: SonarQube

La startup fintech da un feedback del gran trabajo que han realizado, pero María Merengue ejecutiva del área de TI no está contenta debido a que no se ha realizado el análisis estático del código, para eso les encarga una misión y es utilizando SonarQube y Docker les comparte el siguiente paso a paso:

- Descarguen la imagen de docker con el siguiente comando:

```
docker pull sonarqube
```

- Arranquen el servicio de SonarQube con el siguiente comando:

```
docker run -d --name sonarqube -e  
SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true -p 9000:9000  
sonarqube:latest
```

- Validen el funcionamiento con:

```
docker ps -a
```

- Inicien sesión en sonar en el localhost:9000, ahí deben cambiar la clave por defecto, el usuario y contraseña es admin.
- Entren a las opciones de la cuenta: Account -> settings -> generate token
- Una vez sonar este corriendo deben generar el token
- Instale sonarLint en el IDE que se este manejando
- Añadan el plugin de Sonar en el archivo pom del proyecto:

```
<plugin>  
  <groupId>org.sonarsource.scanner.maven</groupId>  
  <artifactId>sonar-maven-plugin</artifactId>  
  <version>4.0.0.4121</version>  
</plugin>
```

- Añadan las

```
<sonar.projectKey>library</sonar.projectKey>  
<sonar.projectName>library</sonar.projectName>  
<sonar.host.url>http://localhost:9000</sonar.host.url>  
<sonar.coverage.jacoco.xmlReportPaths>target/site/jacoco/jacoco.xml</sonar.coverage.jacoco.xmlReportPaths>  
<sonar.coverage.exclusions>src//configurators/*</sonar.coverage.exclusions>
```

propiedades de Sonar y Jacoco:

- Construyan el proyecto, generen el reporte de JACOCO
- Genere la integración con sonar con el comando: `mvn verify sonar:sonar -D sonar.token=[TOKEN_GENERADO]`

Una vez ejecutado realice una captura de pantalla presentando como se ve el análisis estático de su código.

Entregables:



- En el README.md principal, añade una sección para este reto.
- Incluye las evidencias solicitadas en el reto
- Cada uno de los miembros del equipo establezcan una reflexión sobre: ¿Por qué considera que es importante realizar pruebas a un software diseñado? (Integrante que no suba su reflexión en sus palabras no se le vale el reto)